GHOSTBUSTERS / GHOSTOPS

Project Architecture & Implementation Summary

A safety-first FinOps investigation and remediation-assistance platform for Terraform pull requests and cloud-resource discovery.

> **Repository reviewed:** GhostBusters
> **Prepared:** 30 July 2026
> **Scope:** current code, configuration, database schema, frontend, integrations, tests, and deployment assets.

Executive summary

GhostBusters helps FinOps, platform, and engineering teams identify potentially wasteful infrastructure, assemble evidence, evaluate safe alternatives, and route a recommendation through human approval. The product does not apply infrastructure changes. Its safety model is deliberately conservative: destructive or production changes are blocked, missing or conflicting evidence reduces confidence, policy is evaluated deterministically, and a human remains the final decision maker.

The product UI is branded **GhostOps**, while the backend/package/repository remain named **GhostBusters**.

1. Product Scope and Entry Points

There are two complementary workflows that converge on the same safety and review principles.

Terraform PR Review

Analyzes a Terraform plan fixture or an allowlisted GitHub pull request. It understands resource changes, gathers technical and business context, and proposes a safe cost action such as right-sizing or scheduling.

Cloud Hunt

Scans normalized cloud inventory for “ghost” resources: idle, old, unowned, detached, or otherwise suspicious resources. Candidates become review cases, not automatic deletions.

Primary personas

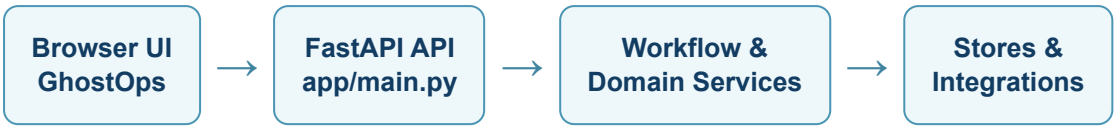
- **Owner/Admin:** manages the workspace, integrations, team membership, schedules, and workspace settings.
- **Reviewer:** inspects evidence and can approve, reject, modify, request evidence, add context, waive, revoke, or reopen within assigned permissions.
- **Viewer:** can inspect organization-scoped results without making decisions.

Implemented product surfaces

The single-page GhostOps frontend contains Overview, Goals, Operations/PR Reviews, Cloud Hunt, Approvals, Settings, outcome verification, technical audit, activity log, demo readiness, authentication, invitations, and the read-only “Ask GhostOps” assistant.

> **Scope boundary:** GhostBusters supports analysis, evidence, policy, review, and carefully guarded PR preparation. It does not run `terraform apply`, merge pull requests, mutate cloud resources, delete branches, or automatically approve a change.

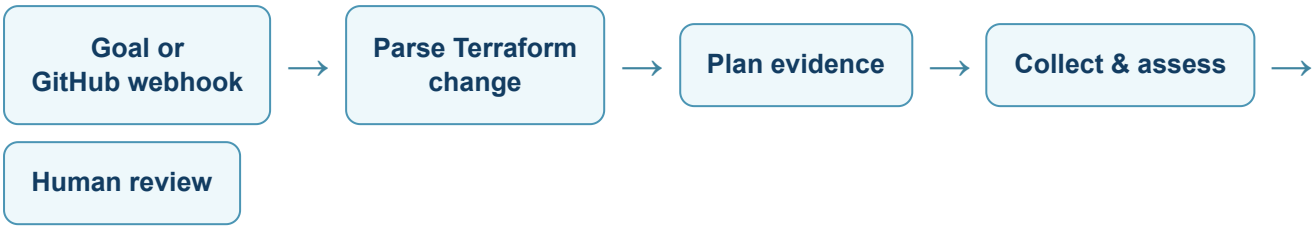
2. High-Level Architecture



Layer	Key implementation	Responsibility
Presentation	Static HTML, CSS, and JavaScript	Responsive application shell, authenticated navigation, dashboards, detailed review screens, settings, and action forms.
API	FastAPI + Pydantic models	HTTP contracts, request validation, endpoint permissions, readiness/liveness, static assets, and webhook intake.
Application services	WorkflowService, CloudHuntService, outcome and assistant services	Orchestrate lifecycle transitions, decisions, persistence, idempotency, and audit events.
Decision engine	Planner, investigator, alternatives, verifier, confidence, policy	Creates explainable deterministic recommendations from structured evidence.
Integration adapters	GitHub, Jira, AWS, cloud registry, Terraform parser	Use narrow, read-only contracts and allowlists; normalize outside data before it reaches the decision engine.
Persistence	PostgreSQL / Redis / in-memory local adapters	Durable workflow truth, audit projections, auth state, review data, deduplication, scheduling locks, and sessions.

Dependency direction is inward: UI and integrations call the API; the API calls application/domain services; services depend on adapters and storage interfaces. This keeps decision rules testable without live cloud, GitHub, Jira, Redis, or database dependencies.

3. Terraform PR Review Workflow

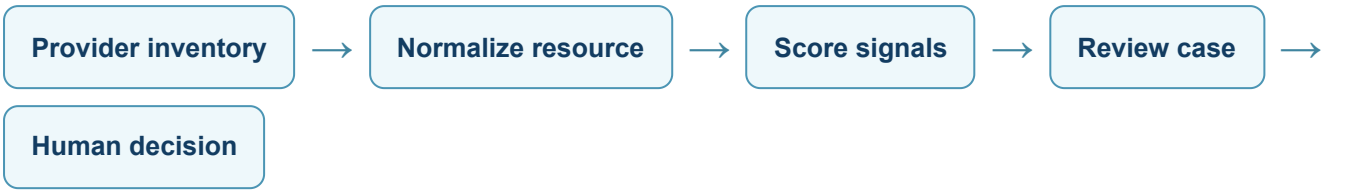


1. **Intake:** a user starts a scenario/goal or GitHub sends a signed pull-request webhook. Delivery IDs and workflow idempotency keys prevent duplicate work.
2. **Terraform interpretation:** plan JSON parsing extracts address, action, before/after configuration, tags, environment, and destructive/replacement signals. GitHub intake restricts parsing to changed `.tf` and `.tf.json` files at the PR head SHA.
3. **Investigation planning:** the deterministic planner selects only relevant evidence tools. Production or destructive cases bypass normal optimization work and move directly into safe policy handling.
4. **Evidence:** pricing, utilization, Jira, Git activity, ownership, dependencies, AWS and GitHub context are represented as structured, timestamped items. Failed collection becomes explicit unavailable evidence.
5. **Decision:** conflicts and missing evidence are identified; alternatives are generated; verifier checks and policy decide whether a recommendation can proceed.
6. **Review:** authorized people approve, reject, modify, request more evidence, or add context. Each action is versioned, idempotent, and audited.

Remediation output

Default behavior creates a simulated/mock remediation PR record. Optional real GitHub PR creation is separately feature-flagged and only supports a narrowly validated size attribute change in a controlled allowlisted repository after human approval.

4. Cloud Hunt Workflow



Cloud Hunt is a second entry path for discovering potentially abandoned resources. The registry uses a provider-neutral `CloudResource` shape, so candidate scoring and review are consistent across AWS, Azure, and GCP fixture sources. An optional real-AWS adapter reads EC2, EBS, Elastic IP, STS, CloudWatch, and AWS Pricing data; Azure and GCP are currently fixture-backed adapters.

Candidate model

Positive signals

Low utilization, age, missing owner, completed work, no recent activity, no dependencies, ongoing cost, unattached volume, or idle public IP.

Protective signals

Production environment, recent activity, active dependencies, or other risk indicators reduce confidence or block automated remediation.

Detected candidates are not changed directly. A case recommends an action such as owner confirmation, observation, or unused-IP release. Resources without a verified Terraform mapping do not receive fabricated Terraform addresses or patches; they receive a proposal to import, route to Jira, or involve the platform owner.

Waivers are implemented: they require an owner/reason/expiry, are visible in the audit trail, and suppress that resource from subsequent hunts until expiry. Scheduling data structures, persistence, and safe coordination primitives exist; automatic scheduled execution remains disabled in production deployment configuration pending a dedicated worker.

5. Decision, Safety, and AI Architecture

Deterministic core

- Generates alternatives including keep, downsize, schedule, request evidence, abstain, and blocked states.
- Calculates confidence from evidence completeness, reliability, freshness, conflicts, and policy certainty.
- Runs independent verifier checks before policy evaluation.
- Uses Rego through Conftest when available, with a fail-safe Python policy fallback if Conftest is missing, disabled, malformed, or times out.

Hard safety controls

Condition	Result
Production resource or destructive/replacement action	Normal optimization is skipped and remediation is blocked.
Unknown ownership, active dependency, critical missing evidence, critical verifier failure, or low confidence	Remediation alternative is denied or escalated for human context.
Unsafe policy-engine execution	Fallback deterministic policy is used; safety is not weakened.
Any remediation action	Human approval remains mandatory.

Optional AI assistance

Gemini or a mock provider may interpret a goal, suggest registered read-only evidence tools, ask a clarifying question, and improve explanations. Every proposal is validated. AI never has authority to parse Terraform, calculate authoritative costs, decide policy, execute tools outside the registry, approve, generate an unchecked patch, merge a PR, or mutate infrastructure. A deterministic fallback preserves the workflow when AI is disabled or unavailable.

6. Security, Identity, and Governance

Workspace and access model

GhostBusters has an organization boundary: users belong to an organization through a membership with Owner, Admin, Reviewer, or Viewer role. Protected routes use centralized permission checks, and workflow runs, Cloud Hunt cases, approvals, waivers, evidence projections, audit rows, and configuration are scoped by `organization_id`.

Authentication

Registration and login use salted scrypt password hashes. Browser sessions are opaque server-side IDs stored in Redis when configured, otherwise in-memory for local development.

Browser protections

HttpOnly, SameSite cookies; CSRF token validation for state-changing actions; generic login errors; request/body limits; and login/expensive-operation throttling.

Invitations

Role-limited invitations are single use and expiring. Only a SHA-256 token hash is persisted. A development link can be returned locally when email delivery is disabled.

Auditability

Workflow events, decisions, integration actions, activity events, and review state changes are append-only/ordered records with actor, correlation, and organization context.

Data protection

Secrets are configured through environment variables; credentials, webhook secrets, database URLs, and sensitive values are redacted before AI-bound payloads and audit metadata. GitHub and Jira use allowlists. AWS onboarding uses a CloudFormation-created customer-side read-only role with external ID, rather than storing customer access keys.

> The production settings validator fails closed when essential configuration is absent: authenticated mode, HTTPS cookies, strong session secret, PostgreSQL, Redis, CORS allowlist, proxy configuration, non-demo pricing, and safe GitHub settings are required.

7. Integrations and Controlled Side Effects

Integration	Implemented capability	Guardrails
GitHub	Webhook HMAC verification; allowlisted PR metadata/files/context; CODEOWNERS, commits, reviews; GitHub App support; optional real remediation PR.	Only selected repositories; supported webhook actions; immutable head SHA; safe paths; real PR flag disabled by default; one supported size attribute; never main/merge/apply.
Jira	Read-only identity/project/issue context, ownership, freshness, lifecycle signals, and Jira-vs-Git conflict detection.	Project allowlist; credentials stay server-side; no Jira mutation.
AWS	Read-only STS validation, EC2/EBS/EIP collection, CloudWatch utilization, and live AWS Pricing lookup with provenance.	Explicit real-AWS selection; regional allowlist; onboarding external ID; no fallback to fixtures; no mutation API.
Azure/GCP	Provider-neutral fixture inventory adapters.	Real credentialed collection is not implemented in this repository.
Terraform CLI	Optional constrained wrapper.	Allows only version, fmt-check, validate, and show-json against supplied data; init/plan/apply/destroy are blocked.

Real PR creation path

When explicitly enabled after approval, the remediation service re-fetches the analyzed file at the recorded head SHA, confirms the expected old value appears exactly once, changes one supported attribute, creates a dedicated `ghostbusters/remediation/...` branch, verifies the diff, and opens a new PR. Validation failure produces no write. Existing GhostBusters PRs are reused to maintain idempotency.

8. Data, Operations, and Deployment

Persistence model

PostgreSQL is the system of record. A workflow run stores a complete JSON snapshot while normalized projections support reporting and audit: evidence records, approvals, audit logs, cloud hunts, review cases, schedules, integration configuration, outcome verifications, activity events, invitations, memberships, organizations, and human decision events. Updates are transactional and use row locking/version increments to avoid stale writes.

Redis supports session storage, webhook delivery deduplication, distributed rate limits, and scheduler coordination. In local development the application can use in-memory run/session adapters; production requires PostgreSQL and Redis.

Closed-loop outcome verification

After an approved recommendation, the platform can open an outcome-verification record, wait for deployment confirmation, record observed cost/utilization/health data, and classify the outcome as verified success, partial, insufficient evidence, regression, or reopened. It intentionally refuses to make an exact savings claim when post-change evidence is incomplete.

Deployment assets

- **Docker Compose:** local PostgreSQL 17 and Redis 7 services.
- **Render blueprint:** FastAPI web service, managed PostgreSQL, Redis, health check, and pre-deploy migration command.
- **Migrations:** baseline plus additive activity/scheduler hardening and canonical organization backfill.
- **Operational endpoints:** `/live` for process liveness and `/ready` for dependency/configuration readiness.

9. Verification, Current Position, and Next Steps

What has been implemented

The repository contains **72 test files and 298 test functions** spanning authentication/RBAC, goal orchestration, deterministic and AI-assisted planning, Terraform parsing, policy, retries, cloud candidate scoring, schedules, AWS/GitHub/Jira integrations, webhooks, real-PR safety paths, persistence, redaction, UI routes, outcomes, and production hardening. The project also contains prepared fixtures for safe, production, destructive, dependency, conflicting, and missing-evidence scenarios.

A local full-suite run initiated for this review exceeded a 60-second execution window before returning a result, so this report does not claim a fresh full-suite pass. The repository documentation should be updated after the next complete CI/local test run to publish a current count and status.

Current maturity assessment

Area	Position
Core workflow and safety logic	Substantially implemented, deterministic, explainable, and test-covered.
User-facing product	Implemented SPA with auth-aware, API-driven operations and settings surfaces.
Real integrations	Opt-in, constrained GitHub/Jira/AWS paths; fixture mode remains available only for explicit demo/test use.
Production readiness	Deployment/configuration hardening is present; production operation still requires environment setup, migration/backup practice, monitoring, and controlled integration rollout.

Recommended next work

1. Run and record a complete CI test result; resolve any slow/hanging integration-test behavior.
2. Operate real GitHub/AWS/Jira integrations only in controlled, allowlisted environments and validate their monitoring/runbooks.
3. Add a dedicated scheduler worker before enabling scheduled Cloud Hunts in production.
4. Expand real Azure/GCP inventory support only with the same read-only, provenance, allowlist, and fail-closed design.
5. Keep the safety boundary explicit in demos: GhostBusters recommends and prepares; normal engineering and CI/CD processes decide and deploy.

> **Bottom line:** GhostBusters is an evidence-driven FinOps copilot with a strong human-in-the-loop safety posture. Its most valuable implementation is not just cost detection—it is the controlled path from a cloud or Terraform signal to an auditable, policy-checked, reviewable recommendation.